

Laboration 2

Från data till slutsatser

Under den första laborationen lärde ni er grunderna i dataanalys. Nu ska vi ta nästa steg.

I verkligheten är dataset sällan perfekta. Dataanalytiker behöver därför kunna:

- upptäcka problem i data,
- hantera saknade värden,
- identifiera avvikande mätningar,
- undersöka samband mellan variabler,
- bygga enkla modeller, och
- dra rimliga slutsatser.

Dagens mål

Målet idag är att ni ska arbeta mer som riktiga dataanalytiker.

Vi kommer inte bara att skapa grafer, utan också försöka förstå vad de faktiskt säger.

Datakvalitet: Första raderna

Innan man börjar analysera ett dataset bör man alltid undersöka dess kvalitet. Om datasetet innehåller problem kan analysen annars bli missvisande.

Vi börjar som vanligt med att titta på de första raderna genom:

```
df.head()
```

Datakvalitet: Snabb överblick

Vi kan också få en snabb överblick över datasetet genom:

```
df.describe()
```

Datakvalitet: Saknade värden

Nu ska vi undersöka om det finns saknade värden:

```
df.isna().sum()
```

Här gäller att:

- `isna()` kontrollerar om ett värde saknas, och
- `sum()` räknar hur många sådana värden som finns.

Sortering av värden

Vi kan även sortera ett dataset för att leta efter extrema värden genom:

```
df.sort_values("variabelnamn")
```

eller

```
df.sort_values("variabelnamn", ascending=False)
```

om vi inte vill ha i stigande ordning.

Saknade värden uppstår ofta i verkliga dataset.

Inom rymddata kan det bland annat bero på:

- moln som blockerar satellitens sikt,
- tekniska problem hos sensorn, och
- fel vid överföring av data.

Hantera saknade värden

Om ett fåtal värden saknas kan man ibland välja att ta bort dem:

```
df = df.dropna()
```

Man kan också välja att ersätta dem med exempelvis medelvärdet:

```
df["variabelnamn"] = df["variabelnamn"].fillna(  
    df["variabelnamn"].mean()  
)
```

Hantera outliers

Liknande metoder kan man använda för att hantera outliers.

Det finns inget universellt rätt metodval, utan det beror på situationen.

Övning del 1

Dags att lösa lite uppgifter!

Filtrering

Ofta vill vi inte analysera hela datasetet samtidigt. Vi kan exempelvis filtrera ut alla observationer från år 2020 och senare genom:

```
df[df["year"] >= 2020]
```

Ofta vill vi inte analysera hela datasetet samtidigt. Vi kan exempelvis filtrera ut alla observationer från år 2020 och senare genom:

```
df[df["year"] >= 2020]
```

Hur koden fungerar är att den för varje rad kollar om variabeln "year" är större än eller lika med 2020.

Om den är det returnar den True, annars False.

Sedan tar den ut alla rader som villkoret returnerar True för.

Filtrering med flera villkor

Vi kan också kombinera flera villkor (i detta fall alla år mellan 2018 och 2023, inklusive) genom:

```
df[(df["year"] >= 2018) & (df["year"] <= 2023)]
```

Detta fungerar på liknande sätt som med enbart ett villkor.

Filtrering är ett viktigt verktyg eftersom olika frågeställningar ofta kräver olika delar av mätvärdena.

Dags att lösa lite uppgifter!

Under föreläsningen pratade vi om korrelation. Nu ska vi undersöka det i praktiken.

Vi börjar med att skapa ett spridningsdiagram genom:

```
df.plot.scatter(x="variabel1", y="variabel2")  
  
plt.show()
```


Pearsons korrelation

Om punkterna verkar följa ett mönster kan det finnas ett samband.

Vi kan beräkna korrelationen numeriskt (där Pearsons korrelation används om man inte specificerar något annat) genom:

```
df["variabel1"].corr(  
    df["variabel2"]  
)
```

Annan korrelation än Pearsons

Vill man ha någon annan korrelation än Pearsons (i detta fall Kendalls) så skriver man:

```
df["variabel1"].corr(  
    method='kendall',  
    df["variabel2"]  
)
```

Dags att lösa lite uppgifter!

Regression

Nu ska vi försöka beskriva ett samband matematiskt.

Vi börjar med linjär regression. Det används för att hitta den linje som bäst beskriver datapunkterna.

För en linjär regression använder vi linregress från biblioteket scipy, så vi börjar med att importera det genom:

```
from scipy.stats import linregress
```

Linjär regression

Sedan utför vi regressionen genom:

```
result = linregress(  
    df["variabel1"],  
    df["variabel2"]  
)
```

Linjär regression

Lutningen fås genom:

```
k = result.slope
```

Skärningen med y-axeln fås genom:

```
m = result.intercept
```

Plotta en linjär regression

För att faktiskt plotta detta måste vi också ha med:

```
x_line = np.linspace(x.min(), x.max(), n)
y_line = k * x_line + m

plt.plot(x_line, y_line)
plt.show()
```

Här skapar `np.linspace` en array med n jämnt utspridda värden från det minsta till det största värdet av x .

På liknande sätt blir y en array med funktionsvärdena av de n x -värdena.

Om sambandet inte verkar linjärt kan vi använda polynomregression, exempelvis av grad 2:

$$y = ax^2 + bx + c.$$

Detta görs med biblioteket numpy, som vi först importerar genom:

```
import numpy as np
```


Polynomregression

Sedan utför vi regressionen genom:

```
coeffs = np.polyfit(  
    df["variabel1"],  
    df["variabel2"],  
    deg=2  
)
```

Vill vi ha en polynomregression av någon annan grad ändrar vi värdet på deg.

Plotta en polynomregression

För att faktiskt plotta detta måste vi också ha med:

```
x_line = np.linspace(x.min(), x.max(), n)
```

```
poly = np.poly1d(coeffs)
```

```
y_line = poly(x_line)
```

```
plt.plot(x_line, y_line)
```

```
plt.show()
```

Plotta en polynomregression

För att faktiskt plotta detta måste vi också ha med:

```
x_line = np.linspace(x.min(), x.max(), n)

poly = np.poly1d(coeffs)
y_line = poly(x_line)

plt.plot(x_line, y_line)
plt.show()
```

Det `np.poly1d` gör är att den gör om koefficienterna till en funktion som vi sedan stoppar in våra x-värden i.

Linjäriserad exponentiell regression

Om vi misstänker ett exponentiellt samband kan vi linjärisera $y = ae^{bx}$ till:

$$\ln(y) = \ln(a) + bx.$$

Genom linjäriseringen kan vi göra en linjär regression, där skärningen med y-axeln motsvarar $\ln(a)$ och lutningen av kurvan motsvarar b .

Notera att y måste vara positivt.

Linjäriserad exponentiell regression

Detta kan vi göra i kod genom:

```
import numpy as np
from scipy.stats import linregress

x = df["variabel1"]
y = df["variabel2"]

y_log = np.log(y)

result = linregress(x, y_log)

b = result.slope
a = np.exp(result.intercept)
```

Plotta en linjäriserad exponentiell regression

För att faktiskt plotta detta måste vi också ha med:

```
x_line = np.linspace(x.min(), x.max(), n)
y_line = a * np.exp(b * x_line)

plt.plot(x_line, y_line)
plt.show()
```

Dags att lösa lite uppgifter!

Det sista steget i dataanalys är inte att skapa fler grafer, utan det är att tolka resultaten.

En dataanalytiker skriver inte: “*Korrelationen är $-0,82$.*”, utan skriver i stället något i stil med: “*Det verkar finnas ett tydligt negativt samband mellan temperatur och vegetation. När temperaturen ökar tenderar NDVI att minska. Vi kan däremot inte avgöra om det är ett orsakssamband utifrån dessa data.*”

Dags att lösa lite uppgifter!

Att göra tydliga grafer

Att skapa en graf är bara första steget.

För att andra ska kunna förstå resultaten behöver grafen också vara tydlig och lätt att tolka.

Några enkla förbättringar kan göra stor skillnad.

En graf bör nästan alltid ha en titel som beskriver vad den visar. Detta görs genom:

```
plt.title("Titeltext")
```

Det ska vara tydligt vad axlarna representerar och vilka enheter som används. Detta görs genom:

```
plt.xlabel("Storhet1 [Enhet1]")  
plt.ylabel("Storhet2 [Enhet2]")
```

Utan axelrubriker kan det vara svårt att förstå vad grafen egentligen visar.

Legend

Om flera dataserier visas i samma graf är det viktigt att ha en legend.
Detta görs genom:

```
plt.plot(year, variabel1, label="storhet1")  
plt.plot(year, variabel2, label="storhet2")  
  
plt.legend()
```

Legenden hjälper läsaren att avgöra vilken kurva som motsvarar vilken variabel.

Begränsa axlarna

Ibland kan det vara användbart att zooma in på en del av datan. Detta görs exempelvis genom:

```
plt.xlim(2015, 2025)
```

```
plt.ylim(0, 100)
```

Detta kan göra trender lättare att upptäcka, men man måste vara försiktig så att man inte skapar en missvisande bild genom att välja olämpliga gränser.

Ett rutnät kan göra det lättare att läsa av värden. Detta skapas genom:

```
plt.grid()
```

När ni arbetar med era egna projekt senare kommer tydliga figurer vara minst lika viktiga som själva analysen.

En bra graf gör det enklare att upptäcka mönster, kommunicera resultat och övertyga andra om era slutsatser.

Dags att lösa lite uppgifter!

Idag har ni genomfört en komplett dataanalys:

Data → Datarensning → Visualisering → Korrelation → Regression →
Slutsats

Detta är samma arbetsprocess som används inom forskning, företag och myndigheter när man arbetar med riktiga data.

Nästa steg blir att börja använda dessa verktyg för att undersöka era egna frågeställningar.